A Modern Merge-Insertion Sort Implementation

Louis Touzalin

August 14, 2026

Abstract

Merge-Insertion Sort, also known as the Ford–Johnson algorithm, is a comparison-based sorting algorithm designed to keep the number of key comparisons close to the information-theoretic minimum. Its classical form first orders disjoint pairs, recursively sorts the larger member of each pair, and then inserts the remaining elements in batches whose boundaries are derived from the Jacobsthal sequence.

This paper separates three notions that are often conflated in discussions of Ford–Johnson: the decision-tree comparison count, the asymptotic running time of a concrete container implementation, and the number of machine instructions. The theoretical part describes the canonical MergeInsertion algorithm and its comparison bound. The implementation part studies a modern C++ variant using a structure-of-arrays pair representation, a comparison-oriented insertion mode, and a hardware-oriented blocked mode with SIMD-assisted scans. The current implementation deliberately replaces the canonical recursive sorting of pair winners with a hybrid merge/insertion sort; therefore its measured comparison count must not be identified with the exact Ford–Johnson worst-case sequence. This distinction is central to the experimental analysis.

Contents

1 Introduction

For a comparison-based sorting algorithm, the input permutation must be identified using pairwise ordering decisions. Any decision tree capable of sorting n distinct elements therefore has at least $n!$ leaves, which gives the information-theoretic lower bound

$$L(n) = \lceil \log_2(n!) \rceil . \tag{1}$$

Using Stirling's approximation,

$$\begin{aligned} \log_2(n!) &= n \log_2 n - (\log_2 e)n + \frac{1}{2}\log_2(2\pi n) + O\left(\frac{1}{n}\right) \\ &= n \log_2 n - 1.442695\ldots\, n + O(\log n). \end{aligned} \tag{2}$$

The coefficient $1.442695\ldots$ is therefore associated with the *lower bound*; it is not the linear coefficient of Ford–Johnson itself.

Ford and Johnson introduced their tournament method in 1959. The algorithm later became widely known as MergeInsertion. Its main interest is that it spends comparisons very carefully: the order in which pending elements are inserted is chosen so that binary searches repeatedly operate on prefixes whose lengths are close to $2^k - 1$.

This paper has two goals. First, it presents the mathematical mechanism behind the classical algorithm without equating it with the information-theoretic lower bound. Second, it documents a modern C++ implementation in which comparison efficiency and machine efficiency are exposed as two distinct optimization targets.

2 Canonical MergeInsertion

2.1 Pair formation

Let the input contain n values. The algorithm first creates $\lfloor n/2 \rfloor$ disjoint pairs. One comparison is sufficient to order each pair:

$$b_i \le a_i, \tag{3}$$

where a_i denotes the larger member and b_i the smaller member.

The pairing phase therefore costs exactly

$$\left\lfloor \frac{n}{2} \right\rfloor \tag{4}$$

logical comparisons for distinct keys.

If n is odd, one value remains unpaired. In the canonical algorithm this value is treated as an additional pending element during the insertion phase.

2.2 Recursive ordering of the winners

The defining recursive step of canonical MergeInsertion is that the a_i values are themselves sorted by MergeInsertion. The pair associations must be preserved, so after this recursive call one has

$$a_1 \le a_2 \le \cdots \le a_m, \qquad b_i \le a_i, \tag{5}$$

with $m = \lfloor n/2 \rfloor$.

The initial main chain is then

$$S = (b_1, a_1, a_2, \ldots, a_m). \tag{6}$$

The remaining b_i values are pending insertions.

2.3 Why the insertion prefix matters

Suppose a value is inserted into a sorted prefix containing q elements. There are $q+1$ possible insertion positions, so a balanced binary decision tree needs at most

$$\lceil \log_2(q+1) \rceil \tag{7}$$

comparisons.

The particularly useful lengths are

$$q = 2^k - 1, \tag{8}$$

because then exactly k comparisons are sufficient in the worst case.

For every pending partner b_i, the relation $b_i \leq a_i$ implies that b_i never needs to be searched to the right of its partner a_i. Ford–Johnson exploits this bound together with a carefully chosen insertion order.

3 Jacobsthal Ordering

3.1 Definition

The Jacobsthal numbers are defined by

$$J_0 = 0, \qquad J_1 = 1, \qquad J_n = J_{n-1} + 2J_{n-2} \quad (n \geq 2). \tag{9}$$

Their closed form is

$$J_n = \frac{2^n - (-1)^n}{3}, \tag{10}$$

hence

$$J_n \sim \frac{2^n}{3}. \tag{11}$$

The relevant batch boundaries for MergeInsertion are

$$t_k = \frac{2^{k+1} + (-1)^k}{3}, \qquad k \geq 1, \tag{12}$$

which gives

$$1,\ 3,\ 5,\ 11,\ 21,\ 43, \ldots \tag{13}$$

These are shifted Jacobsthal numbers.

3.2 The key Jacobsthal identity

The connection between the Jacobsthal sequence and MergeInsertion becomes clearer when two consecutive batch boundaries are added. Starting from

$$t_k = \frac{2^{k+1} + (-1)^k}{3}$$

and

$$t_{k-1} = \frac{2^k + (-1)^{k-1}}{3},$$

we obtain

$$\begin{aligned} t_k + t_{k-1} &= \frac{2^{k+1} + 2^k + (-1)^k + (-1)^{k-1}}{3} \\ &= \frac{3 \cdot 2^k}{3} \\ &= 2^k. \end{aligned}$$

Therefore,

$$\boxed{t_k + t_{k-1} = 2^k}.$$

This identity is the central reason why the Jacobsthal sequence appears in MergeInsertion. The algorithm does not use Jacobsthal numbers merely because they provide an exponentially growing sequence. Rather, consecutive boundaries sum exactly to a power of two, which makes it possible to construct binary search intervals whose maximum useful size is of the form

$$2^k - 1.$$

The relation with the ordinary Jacobsthal sequence is direct. Since

$$J_n = \frac{2^n - (-1)^n}{3},$$

we have

$$t_k = \frac{2^{k+1} + (-1)^k}{3} = \frac{2^{k+1} - (-1)^{k+1}}{3} = J_{k+1}.$$

Hence the Ford–Johnson batch boundaries are simply shifted Jacobsthal

3.3 Insertion batches

The first pending element b_1 is already placed at the beginning of the main chain. The other pending elements are inserted in descending batches:

$$b_3, b_2, \quad b_5, b_4, \quad b_{11}, b_{10}, b_9, b_8, b_7, b_6, \quad \dots \tag{14}$$

More generally, for consecutive Jacobsthal boundaries $t_{k-1} < t_k$, the batch is

$$b_{t_k}, b_{t_k-1}, \dots, b_{t_{k-1}+1}. \tag{15}$$

Indices larger than the number of pending elements are simply truncated.

The important point is not that Jacobsthal numbers "spread insertions uniformly". Their role is more precise: this batch structure controls the largest searchable prefix before each insertion and repeatedly places that prefix close to a binary-search threshold $2^r - 1$.

3.4 Why the batches minimize binary-search depth

The significance of the descending insertion order can now be derived from the pair invariant

$$b_i \le a_i.$$

Since a_i is already present in the sorted main chain when b_i is inserted, it is unnecessary to search for b_i anywhere to the right of a_i. The search interval is therefore bounded by the current position of its associated winner.

Consider the batch whose upper boundary is t_k. Before this batch begins, all pending elements up to index t_{k-1} have already been processed. For the first element of the new batch, b_{t_k}, there are at most t_k winner positions together with the previously inserted pending elements before its associated winner. Consequently, the largest relevant binary-search prefix contains at most

$$t_k + t_{k-1} - 1$$

elements.

Using the Jacobsthal identity

$$t_k + t_{k-1} = 2^k,$$

this becomes

$$t_k + t_{k-1} - 1 = 2^k - 1.$$

A sorted sequence containing at most $2^k - 1$ elements has at most 2^k possible insertion positions. Therefore a binary decision tree of depth k is sufficient:

$$\left\lceil \log_2 \left((2^k - 1) + 1 \right) \right\rceil = k.$$

Thus every insertion in this batch can be arranged so that its worst-case binary-search depth is at most k.

The descending order

$$b_{t_k}, b_{t_k - 1}, \ldots, b_{t_{k-1}+1}$$

is important. After one pending element is inserted, the next element belongs to a pair with a winner located no farther to the right. The growth caused by the previous insertion is therefore compensated by moving to an earlier partner. The relevant search prefix remains bounded by the same $2^k - 1$ threshold throughout the batch.

This explains the algorithmic role of Jacobsthal ordering: it groups pending elements so that several consecutive insertions can all be performed within the same optimal binary-search depth.

4 Comparison Complexity

4.1 Information-theoretic lower bound

Any deterministic comparison-based sorting algorithm can be represented by a binary decision tree. Each internal node corresponds to a comparison between two keys, while each leaf represents one possible ordering of the input. For n distinct elements, there are $n!$ possible permutations, and therefore the decision tree must contain at least $n!$ leaves. A binary tree of height h contains at most 2^h leaves. Consequently, any comparison sorting algorithm must satisfy

$$2^h \geq n!.$$

Taking the base-two logarithm gives

$$h \geq \log_2(n!).$$

Since the number of comparisons is an integer, the worst-case comparison count is bounded below by

$$L(n) = \lceil \log_2(n!) \rceil .$$

This result is independent of any particular sorting strategy: it is a lower bound on every algorithm whose only source of information about the ordering is the result of pairwise comparisons. Using Stirling's approximation,

$$n! = \sqrt{2\pi n} \left(\frac{n}{e} \right)^n \left(1 + O\left(\frac{1}{n} \right) \right),$$

we obtain

$$\begin{aligned} \log_2(n!) &= n \log_2 n - n \log_2 e + \frac{1}{2} \log_2(2\pi n) + O\left(\frac{1}{n} \right) \\ &= n \log_2 n - 1.442695\ldots\, n + O(\log n). \end{aligned}$$

The coefficient $1.442695\ldots$ therefore belongs to the information-theoretic lower bound. It must not be interpreted as the linear coefficient of the Ford–Johnson comparison count itself.

$$L(n) = \lceil \log_2(n!) \rceil . \tag{16}$$

It is impossible for a comparison sort to have a worst-case decision tree shallower than this bound.

4.2 Canonical Ford–Johnson count

The exact worst-case comparison count of canonical MergeInsertion can be written as

$$C_{\mathrm{FJ}}(n) = \sum_{k=1}^{n} \left\lceil \log_2\left(\frac{3k}{4}\right) \right\rceil . \tag{17}$$

For the first input sizes this gives

n	1	2	3	4	5	6	7	8	9	10
$C_{\mathrm{FJ}}(n)$	0	1	3	5	7	10	13	16	19	22

(18)

The asymptotic behavior should not be confused with the lower bound. MergeInsertion has

$$C_{\mathrm{FJ}}(n) = n \log_2 n + b(n)n + O(\log n), \tag{19}$$

where the linear coefficient is oscillatory. Modern analyses place $b(n)$ roughly between -1.415 and -1.329, depending on n. Consequently, Ford–Johnson is extremely comparison-efficient but does not generally attain $\log_2(n!)$.

4.3 Recurrence and batch cost

The comparison count can also be derived directly from the recursive structure of the algorithm. Let $C(n)$ denote the worst-case number of comparisons required by canonical MergeInsertion on n distinct elements. The initial pairing phase requires

$$\left\lfloor \frac{n}{2} \right\rfloor$$

comparisons. The larger elements of the pairs are then sorted recursively, which contributes

$$C\left(\left\lfloor \frac{n}{2} \right\rfloor\right).$$

Let

$$p = \left\lceil \frac{n}{2} \right\rceil$$

denote the number of pending elements when an unpaired element, if present, is included. The element b_1 is placed directly at the beginning of the main chain and therefore requires no insertion comparison. For $k \geq 2$, the k-th Jacobsthal batch contains

$$t_k - t_{k-1}$$

elements when it is complete. If the last batch is truncated because $p < t_k$, the actual number of elements in that batch is

$$B_k(n) = \max\left(0,\, \min(p, t_k) - t_{k-1}\right).$$

From the previous section, every element of this batch requires at most k binary comparisons. Hence the total insertion cost is

$$I(n) = \sum_{k \geq 2} k\, B_k(n),$$

or explicitly,

$$I(n) = \sum_{k \geq 2} k\, \max\left(0,\, \min\left(\left\lceil \frac{n}{2} \right\rceil, t_k\right) - t_{k-1}\right).$$

The complete worst-case recurrence is therefore

$$\boxed{C(n) = \left\lfloor \frac{n}{2} \right\rfloor + C\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + I(n)}$$

with

$$C(0) = C(1) = 0.$$

Evaluating this recurrence yields the classical Ford–Johnson comparison sequence

$$0, 1, 3, 5, 7, 10, 13, 16, 19, 22, \ldots$$

for

$$n = 1, 2, \ldots, 10.$$

An equivalent compact expression is

$$\boxed{C_{\mathrm{FJ}}(n) = \sum_{j=1}^{n} \left\lceil \log_2 \left(\frac{3j}{4} \right) \right\rceil}.$$

The summation form hides the recursive construction but is convenient for computing the exact worst-case count. The recurrence, by contrast, makes clear where the comparisons originate: pair formation, recursive winner sorting, and Jacobsthal-bounded insertion.

4.4 Complete example for $n = 8$

Consider now a complete worst-case comparison count for $n = 8$. The eight input values are first divided into four pairs. Ordering these pairs requires exactly

$$\frac{8}{2} = 4$$

comparisons. The four larger elements

$$a_1, a_2, a_3, a_4$$

must then be sorted recursively. Canonical MergeInsertion requires

$$C(4) = 5$$

comparisons in the worst case. After this recursive step, the main chain begins as

$$S = (b_1, a_1, a_2, a_3, a_4).$$

There are four pending elements in total,

$$p = \left\lceil \frac{8}{2} \right\rceil = 4,$$

but b_1 is already contained in the main chain. The first Jacobsthal batch has

$$t_1 = 1, \qquad t_2 = 3,$$

and therefore contains

$$b_3, b_2.$$

Since

$$t_2 + t_1 - 1 = 3 + 1 - 1 = 3 = 2^2 - 1,$$

each of these two elements can be inserted using at most

$$2$$

comparisons. The first batch therefore costs

$$2 + 2 = 4$$

comparisons. The next Jacobsthal boundary is

$$t_3 = 5.$$

Since only four pending elements exist, this batch is truncated and contains only

$$b_4.$$

Its search interval is bounded by the next binary-search threshold

$$2^3 - 1 = 7,$$

so it requires at most

$$3$$

comparisons. The complete insertion cost is therefore

$$I(8) = 2 + 2 + 3 = 7.$$

Finally,

$$\begin{aligned} C(8) &= \underbrace{4}_{\text{pairing}} + \underbrace{C(4)}_{\text{recursive winners}} + \underbrace{7}_{\text{pending insertions}} \\ &= 4 + 5 + 7 \\ &= \boxed{16}. \end{aligned}$$

This agrees with the exact Ford–Johnson formula:

$$\begin{aligned} C_{\mathrm{FJ}}(8) &= \sum_{j=1}^{8} \left\lceil \log_2 \left(\frac{3j}{4} \right) \right\rceil \\ &= 0 + 1 + 2 + 2 + 2 + 3 + 3 + 3 \\ &= \boxed{16}. \end{aligned}$$

This example illustrates the entire mechanism of the algorithm in a single case: pairwise ordering reduces the problem size, recursive MergeInsertion orders the winners, and Jacobsthal batching controls the depth of the remaining binary searches.

4.5 Comparison count versus running time

The comparison count is only one cost model. A concrete implementation has additional costs:

- moving elements inside contiguous containers;
- memory allocation and capacity growth;
- maintenance of pair associations;
- branch prediction and instruction dependencies;
- cache and TLB behavior;
- SIMD packing, loads, stores and masks.

For example, locating an insertion position by binary search costs $O(\log n)$ comparisons, while inserting into the middle of a `std::vector` may move $O(n)$ values. Repeating such insertions can therefore lead to $O(n^2)$ element movement even though the decision-tree comparison count remains $\Theta(n \log n)$.

The three quantities

$$\text{logical comparisons}, \qquad \text{machine instructions}, \qquad \text{wall-clock time} \tag{20}$$

must therefore be reported separately.

5 Current C++ Implementation

5.1 Implementation status

The current code implements the Ford–Johnson pair formation and Jacobsthal insertion mechanism, but it is deliberately not a literal implementation of the canonical recursive algorithm.

In particular:

- the pair winners are sorted by a hybrid merge/insertion sort rather than recursively by MergeInsertion;
- the `MINCMP` mode respects the partner bound during insertion;
- the `FAST` mode instead uses a blocked sorted chain and hardware-oriented searching;
- an odd straggler is currently inserted after the partner insertions, rather than being integrated into the canonical Jacobsthal pending schedule.

For this reason, the implementation is best described as a *Ford–Johnson-based MergeInsertion variant*. Its comparison counter is useful for evaluating the implementation, but its worst-case values are not the canonical Ford–Johnson sequence.

5.2 Structure-of-arrays pair representation

The pair data are represented by two contiguous arrays:

```
struct PairSoA {
    std::vector<int> lower;
    std::vector<int> upper;

    explicit PairSoA(size_t pair_count)
        : lower(pair_count), upper(pair_count) {
    }

    size_t size() const {
        return lower.size();
    }
};
```

Listing 1: Structure-of-arrays representation for ordered pairs.

For a contiguous `std::vector<int>` input, the pair construction can use NEON to process four pairs at once:

```
PairSoA build_pair_soa(const std::vector<int>& values) {
    const size_t pair_count = values.size() / 2;
    PairSoA pairs(pair_count);
    size_t pair_index = 0;
    size_t value_index = 0;

#if PMERGEME_SIMD_NEON
    for (; value_index + 8 <= values.size();
         value_index += 8, pair_index += 4) {
        const int32x4x2_t interleaved =
            vld2q_s32(values.data() + value_index);
        const int32x4_t lower =
            vminq_s32(interleaved.val[0], interleaved.val[1]);
        const int32x4_t upper =
            vmaxq_s32(interleaved.val[0], interleaved.val[1]);

        vst1q_s32(pairs.lower.data() + pair_index, lower);
        vst1q_s32(pairs.upper.data() + pair_index, upper);
        comparison_count += 4;
    }
#endif

    for (; pair_index < pair_count;
         ++pair_index, value_index += 2) {
        normalize_pair_scalar(values[value_index],
                              values[value_index + 1],
                              pairs.lower[pair_index],
                              pairs.upper[pair_index]);
        ++comparison_count;
    }

    return pairs;
}
```

Listing 2: SIMD-assisted pair normalization.

Here `comparison_count` models logical key comparisons, not the number of emitted CPU comparison instructions. SIMD can reduce instruction overhead or improve throughput, but it does not make eight independent ordering relations count as one comparison in the decision-tree model.

5.3 Jacobsthal generation

The former attempt to compute several recurrence-dependent Jacobsthal values in parallel has been removed. The current implementation uses a lookup table and then the scalar recurrence:

```
std::vector<uint64_t>
generate_jacobsthal_sequence(size_t size) {
    const size_t static_size =
        sizeof(jacobsthal_table) /
        sizeof(jacobsthal_table[0]);

    const size_t limit = std::min(size, static_size);
    std::vector<uint64_t> jacobsthal(size);

    for (size_t i = 0; i < limit; ++i)
        jacobsthal[i] = jacobsthal_table[i];

```

```
    for (size_t i = static_size; i < size; ++i)
        jacobsthal[i] =
            jacobsthal[i - 1] +
            2 * jacobsthal[i - 2];

    return jacobsthal;
}
```

Listing 3: Correct Jacobsthal generation.

This is appropriate because the recurrence contains a loop-carried dependency: J_i is required to compute later terms.

5.4 Building the Ford–Johnson insertion order

The current implementation now constructs descending batches rather than inserting only at isolated Jacobsthal indices:

```
static std::vector<size_t>
build_jacobsthal_insert_order(size_t pair_count) {
    std::vector<size_t> order;

    if (pair_count <= 1)
        return order;

    const std::vector<uint64_t> jacobsthal =
        generate_jacobsthal_sequence(pair_count + 2);

    size_t previous = 1;
    order.reserve(pair_count - 1);

    for (size_t jacob_index = 3;
         previous < pair_count;
         ++jacob_index) {
        const size_t current =
            static_cast<size_t>(
                std::min<uint64_t>(
                    jacobsthal[jacob_index],
                    pair_count));

        for (size_t index = current;
             index > previous;
             --index)
            order.push_back(index - 1);

        if (current == pair_count)
            break;

        previous =
            static_cast<size_t>(
                jacobsthal[jacob_index]);
    }

    return order;
}
```

Listing 4: Construction of the Jacobsthal batch order.

For sufficiently many pairs this produces the zero-based equivalent of

$$b_3, b_2,\ b_5, b_4,\ b_{11}, b_{10}, \ldots, b_6, \ldots$$

6 Comparison-Oriented Mode

6.1 Bounded binary insertion

The `PMERGEME_MODE_MINCMP` path performs a true binary search inside a bounded prefix:

```
static size_t
find_insertion_position_binary(
    const std::vector<int>& chain,
    size_t size,
    int value) {

    size_t left = 0;
    size_t right = size;

    while (left < right) {
        const size_t middle =
            left + ((right - left) / 2);

        ++comparison_count;
        if (chain[middle] < value)
            left = middle + 1;
        else
            right = middle;
    }

    return left;
}
```

Listing 5: Binary insertion position with explicit comparison accounting.

The search bound is the current position of the corresponding winner a_i, not the length of the complete chain.

6.2 Tracking moving winner positions

Every insertion before an a_i shifts its position. The implementation tracks these shifts with a Fenwick tree:

```
class FenwickShifts {
public:
    explicit FenwickShifts(size_t size)
        : tree_(size + 1, 0) {
    }

    void add_suffix(size_t start, int delta) {
        if (start >= size())
            return;

        for (size_t index = start + 1;
             index < tree_.size();
             index += lowbit(index))
            tree_[index] += delta;
    }

    int point_query(size_t index) const {
        int sum = 0;

        for (size_t current = index + 1;
             current > 0;
             current -= lowbit(current))
            sum += tree_[current];

        return sum;
    }

    size_t size() const {
        return tree_.size() - 1;
    }

private:
    static size_t lowbit(size_t index) {
        return index & (~index + 1);
    }

    std::vector<int> tree_;
};
```

Listing 6: Fenwick tree used to track insertion-induced shifts.

The current position of a_i is then

```
static size_t
current_upper_position(const FenwickShifts& shifts, size_t upper_index) {

    return upper_index + 1 + static_cast<size_t(shifts.point_query(upper_index));
}
```

Listing 7: Current winner position.

Finally, insertion of a pending partner is bounded by that position:

```
static void
insert_partner_low(
    std::vector<int>& chain,
    FenwickShifts& shifts,
    size_t pair_index,
    int value) {

    const size_t bound = current_upper_position(shifts, pair_index);
    const size_t position = find_insertion_position_binary(chain, bound, value);
    insert_into_chain(chain, position, value);

    shifts.add_suffix(find_first_affected_upper(shifts, position),1);
}
```

Listing 8: Partner-bounded insertion in MINCMP mode.

This is the essential Ford–Johnson invariant implemented correctly in the current `MINCMP` insertion phase:

$$b_i \le a_i \quad \Longrightarrow \quad \text{search}(b_i) \subseteq S[0, \text{pos}(a_i)). \tag{21}$$

7 Hardware-Oriented Mode

7.1 Blocked sorted chain

The `PMERGEME_MODE_FAST` path intentionally optimizes a different objective. Instead of preserving the minimum-comparison search structure, it stores the sorted chain in small contiguous blocks:

```
void insert(int value) {
    if (blocks_.empty()) {
        std::vector<int> block;
        block.reserve(kBlockMaxSize);
        block.push_back(value);
        blocks_.push_back(block);
        size_ = 1;
        return;
    }

    const size_t block_index = find_block(value);
    std::vector<int>& block = blocks_[block_index];
    const size_t position = find_insertion_position(block, block.size(), value);
    const size_t old_size = block.size();
    block.push_back(value);
    if (position < old_size)
        std::memmove(block.data() + position + 1,
            block.data() + position, (old_size - position) * sizeof(int));

    block[position] = value;
    ++size_;

    if (block.size() > kBlockMaxSize)
        split_block(block_index);
}
```

Listing 9: Core insertion operation of the blocked fast path.

The block containing the insertion point is first selected by binary search on block maxima. Inside a block, the implementation may use AVX2 or NEON to scan several integers at once.

7.2 SIMD scan

On AVX2, eight values are compared in parallel:

```
const __m256i vector_value = _mm256_set1_epi32(value);

for (; i + 8 <= size; i += 8) {
    const __m256i data = _mm256_loadu_si256(reinterpret_cast<
                const __m256i*>(arr.data() + i));

    const __m256i gt = _mm256_cmpgt_epi32(data, vector_value);
    const __m256i eq = _mm256_cmpeq_epi32(data, vector_value);
    const __m256i ge = _mm256_or_si256(gt, eq);
    const int mask = _mm256_movemask_ps(_mm256_castsi256_ps(ge));
    comparison_count += 8;

    if (mask != 0)
        return i + static_cast<size_t>( __builtin_ctz(static_cast<unsigned int>(mask)
    ));
}
```

Listing 10: AVX2 scan used by the fast insertion path.

This path may reduce elapsed time by improving throughput and locality, but it usually performs more logical comparisons than a binary insertion. That is expected: `FAST` and `MINCMP` deliberately optimize different cost functions.

8 Winner Sorting and Deviation from Canonical Ford–Johnson

The current implementation sorts pair winners with a hybrid merge/insertion sort:

```
static void
merge_sort_pairs_by_upper(PairSoA& pairs, PairSoA& buffer, size_t left, size_t right)
     {

    static const size_t threshold = 32;

    if (left >= right)
        return;

    if (right - left < threshold) {
        insertion_sort_pairs(
            pairs, left, right);
        return;
    }

    const size_t middle =
        left + ((right - left) / 2);

    merge_sort_pairs_by_upper(
        pairs, buffer, left, middle);

    merge_sort_pairs_by_upper(
        pairs, buffer,
        middle + 1, right);

    merge_pairs_by_upper(
        pairs, buffer,
        left, middle, right);
}
```

Listing 11: Current winner-sorting strategy.

This is a practical engineering choice, but it changes the comparison recurrence. Canonical Ford–Johnson would recursively apply MergeInsertion to the winners. Therefore the comparison analysis of the implementation must be written as

$$C_{\text{impl}}(n) = \left\lfloor \frac{n}{2} \right\rfloor + C_{\text{winner}}(\lfloor n/2 \rfloor) + C_{\text{insert}}(n), \tag{22}$$

where C_{winner} is the comparison cost of the current hybrid winner sort. It is not valid to substitute $C_{\text{FJ}}(\lfloor n/2 \rfloor)$ unless that recursive implementation is actually used.

There is a second source of additional logical comparisons in the current winner comparator:

```
static inline bool pair_less(const PairSoA& pairs, size_t lhs, size_t rhs) {

    ++comparison_count;
    if (pairs.upper[lhs] < pairs.upper[rhs])
        return true;

    ++comparison_count;
    if (pairs.upper[rhs] < pairs.upper[lhs])
        return false;

    ++comparison_count;
    return
        pairs.lower[lhs] <
        pairs.lower[rhs];
}
```

Listing 12: Current total-order comparator for pairs.

For distinct values, canonical comparison accounting only needs to compare the winner keys. The additional comparisons above provide deterministic tie-breaking for duplicate keys but increase the measured count.

9 Odd Input Sizes

The current command-line front end removes an odd final value before pair formation:

```
if ((n % 2) != 0) {
    has_straggler = true;
    straggler = data.back();
    data.pop_back();
    data_deque.pop_back();
    --n;
}
```

Listing 13: Current extraction of an odd straggler.

In the present implementation, that value is inserted after the partner batches:

```
if (has_straggler) {
    const size_t position =
        find_insertion_position_binary(
            chain,
            chain.size(),
            straggler);

    insert_into_chain(
        chain,
        position,
        straggler);
}
```

Listing 14: Current MINCMP straggler insertion.

This is correct as a sorting operation, but it differs from canonical MergeInsertion, where the unpaired value belongs to the pending sequence and participates in the Jacobsthal insertion schedule. This difference can add comparisons for odd n.

10 Measured Comparison Counts of the Current MINCMP Variant

To validate the distinction between the canonical algorithm and the current implementation, all permutations were exhaustively tested for $1 \leq n \leq 10$. The following table compares the observed worst-case logical comparison count of the current `MINCMP` implementation with the canonical Ford–Johnson sequence.

n	1	2	3	4	5	6	7	8	9	10
Current MINCMP	0	1	3	6	9	13	16	23	27	35
Canonical FJ	0	1	3	5	7	10	13	16	19	22

Table 1: Worst-case logical comparisons for all permutations up to $n = 10$. The gap is expected because winner sorting and odd-element handling are not yet canonical Ford–Johnson.

The table should not be interpreted as a failure of the Jacobsthal insertion logic. The current implementation correctly constructs descending Jacobsthal batches and, in `MINCMP` mode, bounds each partner insertion by its winner. Most of the remaining discrepancy is architectural: winner sorting does not recursively use MergeInsertion, tie-breaking introduces extra comparisons, and the odd straggler is inserted separately.

11 Benchmarking Methodology

Runtime benchmarking must be kept separate from the decision-tree comparison experiment. For reproducible timing results, each benchmark should report at least:

- CPU model and memory configuration;
- operating system;
- compiler and exact version;
- optimization flags, including architecture flags;
- selected mode: `FAST` or `MINCMP`;
- input distribution and random seed;
- number of warm-up runs;
- number of measured runs;
- median and a dispersion statistic such as standard deviation or interquartile range.

The previous timing tables are intentionally not reused here because they were produced by an older implementation and would not constitute measurements of the code documented in this revision.

The current program already exposes useful comparison statistics:

```
const long double lower_bound =
    (data_count > 1)
    ? (std::lgammal(n + 1.0L)
       / std::log(2.0L))
    : 0.0L;

std::cout
    << "comparisons / (n log2 n) = "
    << normalized_n_log_n
    << std::endl;

std::cout
    << "comparisons / log2(n!) = "
    << lower_bound_ratio
    << std::endl;
```

Listing 15: Comparison statistics reported by the current program.

For a decision-tree comparison, the integral lower bound

$$\lceil \log_2(n!) \rceil \tag{23}$$

should additionally be displayed.

A single-input "empirical exponent" $\log C(n)/\log n$ is not a reliable complexity estimator for a function of the form $n \log n$. The normalized quantity

$$\frac{C(n)}{n \log_2 n} \tag{24}$$

or a fit across several values of n is more informative.

12 `std::vector` and `std::deque`

The current `std::deque` entry point is a compatibility wrapper:

```
std::deque<int>
ford_johnson_sort_deque(
    const std::deque<int>& values,
    int straggler,
    bool has_straggler) {

    reset_comparison_count();

    const std::vector<int> sorted =
        ford_johnson_sort_soa(
            build_pair_soa(values),
            straggler,
            has_straggler);

    return std::deque<int>(
        sorted.begin(),
        sorted.end());
}
```

Listing 16: Current deque wrapper.

Consequently, a timing comparison labelled "vector sort versus deque sort" would be misleading: after pair construction, both paths use the same vector-based core. A meaningful container comparison would require two independent storage backends or should explicitly state that only input construction and output conversion differ.

13 Discussion

The revised implementation exposes a useful engineering trade-off.

The `MINCMP` mode attempts to preserve the principal comparison-saving invariant of MergeInsertion: each b_i is inserted only into the prefix ending at its associated a_i. The Fenwick tree makes the moving partner positions inexpensive to recover after previous insertions.

The `FAST` mode instead targets machine behavior. It partitions the main chain into bounded contiguous blocks, searches block maxima, and uses SIMD-assisted scans inside the selected block. This can reduce data movement and improve cache locality at the cost of additional logical comparisons.

These modes should not be ranked using a single metric. A robust evaluation should report both

$$C(n) \tag{25}$$

for logical key comparisons and

$$T(n) \tag{26}$$

for elapsed execution time.

The implementation can be brought closer to canonical Ford–Johnson by three further changes:

1. recursively sorting pair winners with MergeInsertion while preserving partner associations;
2. integrating the odd straggler into the pending Jacobsthal schedule;
3. separating duplicate-key tie-breaking comparisons from the canonical distinct-key comparison metric.

14 Conclusion

MergeInsertion is notable because it optimizes a specific resource: *comparisons*. Its efficiency comes from a combination of pairwise ordering, recursive winner sorting, partner-bounded binary searches and Jacobsthal-organized insertion batches. The information-theoretic lower bound $\lceil \log_2(n!) \rceil$ provides the reference point, but it must not be identified with the Ford–Johnson comparison count itself.

The current C++ implementation correctly incorporates the pair structure, Jacobsthal batch construction and partner-bounded insertion in `MINCMP` mode. It also offers a separate `FAST` path designed around blocked storage and SIMD-assisted scans. Because winner sorting and odd element handling still differ from canonical MergeInsertion, the measured comparison sequence is intentionally reported as an implementation result rather than as the Ford–Johnson optimum.

This separation between algorithmic comparison efficiency and hardware efficiency is the main design principle of the revised implementation and provides a clean basis for future work.

References

[1] L. R. Ford, Jr. and S. M. Johnson, *A Tournament Problem*, The American Mathematical Monthly, vol. 66, no. 5, pp. 387–389, 1959.

[2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998.

[3] F. Stober and A. Weiss, *On the Average Case of MergeInsertion*, Algorithmica, 2020. Preprint available at https://arxiv.org/abs/1905.09656.

[4] G. K. Manacher, *The Ford–Johnson Sorting Algorithm Is Not Optimal*, Journal of the ACM, vol. 26, no. 3, pp. 441–456, 1979.

[5] M. Ayala-Rincon and B. T. de Abreu, *A Variant of the Ford–Johnson Algorithm that is More Space Efficient*, Information Processing Letters, vol. 102, no. 5, pp. 201–207, 2007.

[6] K. Iwama and J. Teruyama, *Improved Average Complexity for Comparison-Based Sorting*, 2017. Preprint available at https://arxiv.org/abs/1705.00849.